Cover Page

Lab 8	Attacking Passwords
Duration	3 hours

Module	Professional Penetration Testing
Instructor	Phillip Fitzpatrick

Student Name	Danyil Tymchuk	Student ID	B00167321

Date	23/03/2026	Submission Date	30/03/2026

Table of Contents

Table of Contents 2
University Declaration & Use of AI 3
Lab 8: Attacking Passwords 4

University Declaration & Use of AI

University Declaration
I confirm that this submission is my own work, except where clearly indicated. I have acknowledged all sources used. I understand the university's policies on academic integrity and plagiarism.

Use of AI Tools Declaration
I had not used AI at any stage of this assessment.

Signed: Danyil Tymchuk
Date: 23/03/2026

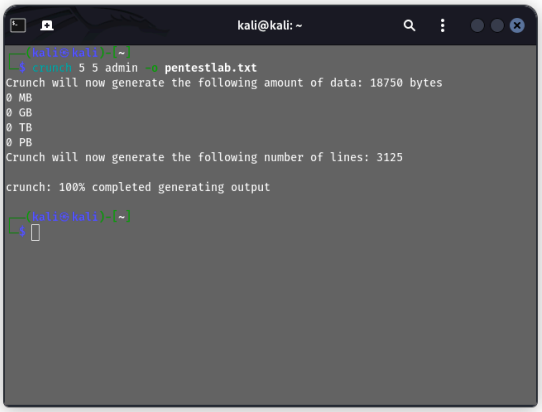
Lab 8: Attacking Passwords

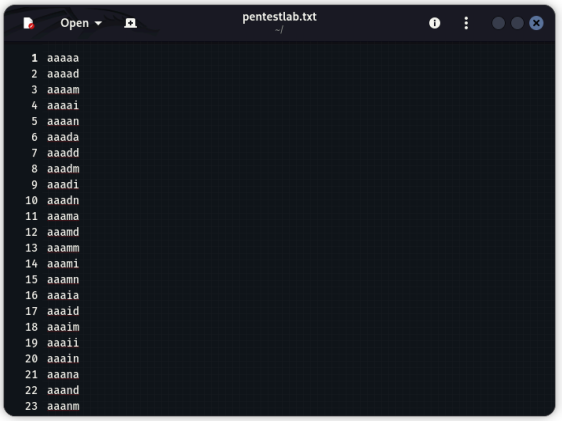
- Using Crunch to generate custom wordlists
- Learning to use Hydra
- Cracking windows and linux password hashes
- Cracking windows LM Hashes and SAM file
- Introduction to Hashcat
- THM hash cracking rooms

Using Crunch to generate custom wordlists

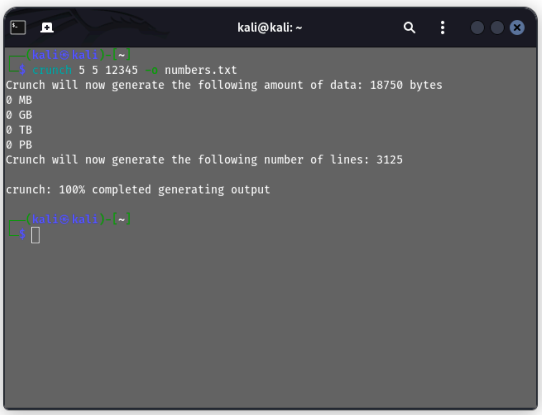
Create a Sample Wordlist

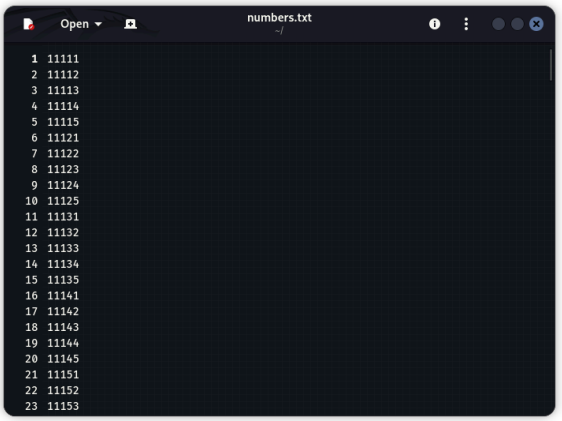
```
crunch 5 5 admin -o pentestlab.txt
```



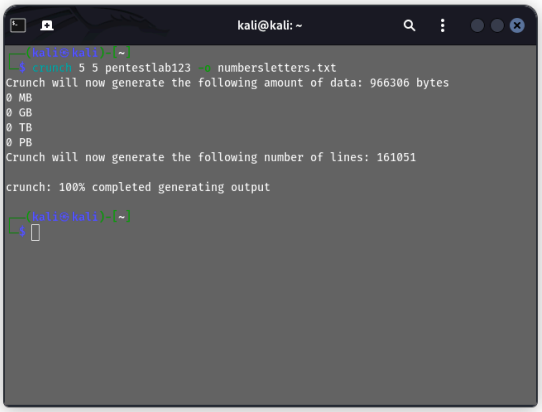


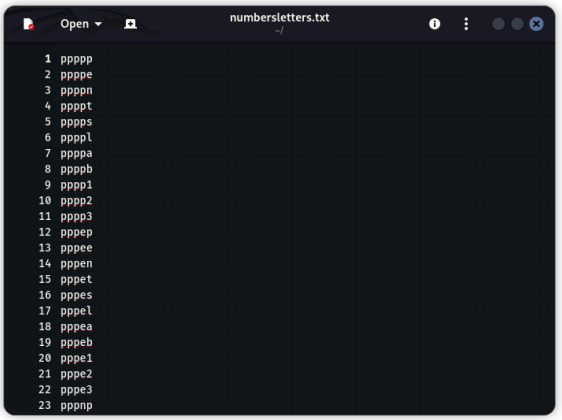

```
crunch 5 5 12345 -o numbers.txt
```





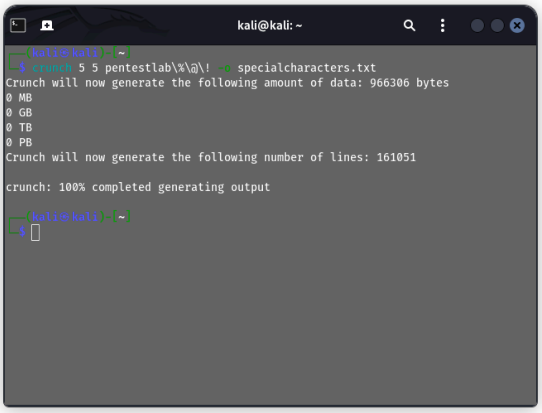
```
crunch 5 5 pentestlab123 -o numbersletters.txt
```

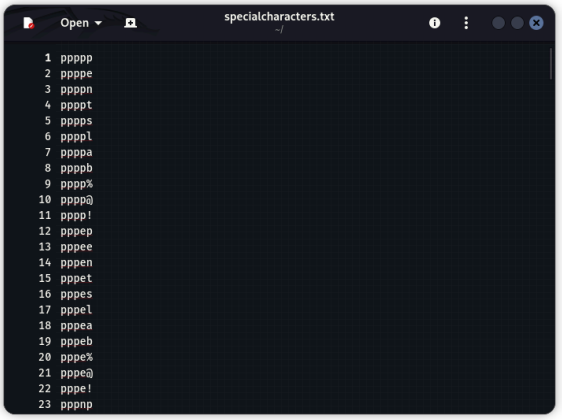




Special Characters

```
crunch 5 5 pentestlab\%\@\! -o specialcharacters.txt
```

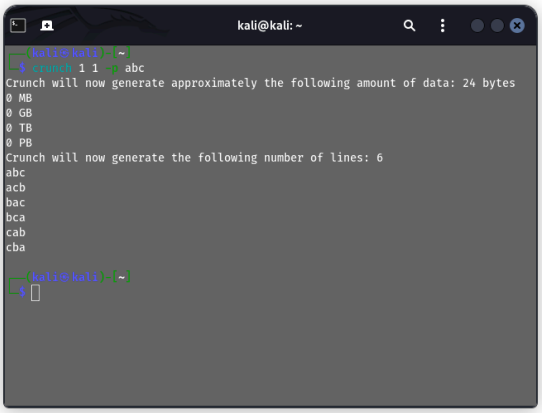


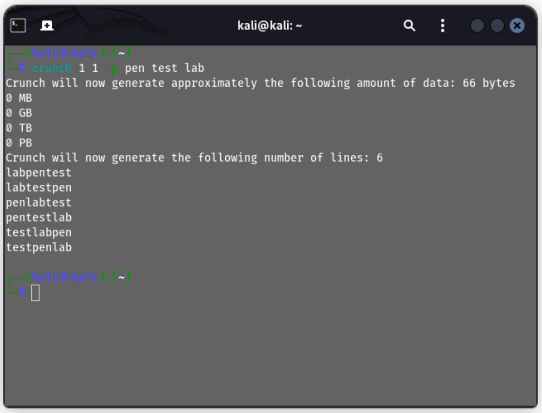


String Permutations

```
crunch 1 1 -p abc
```

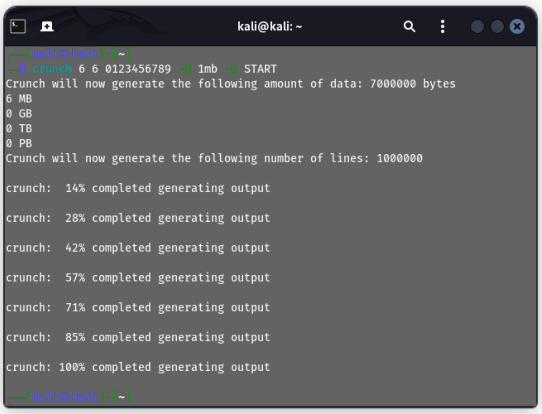
```
crunch 1 1 -p pen test lab
```

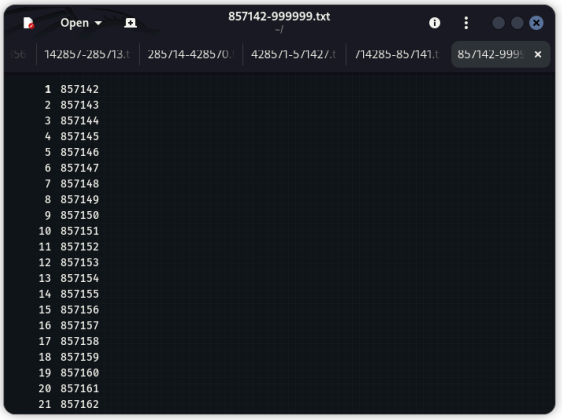




Splitting Wordlists

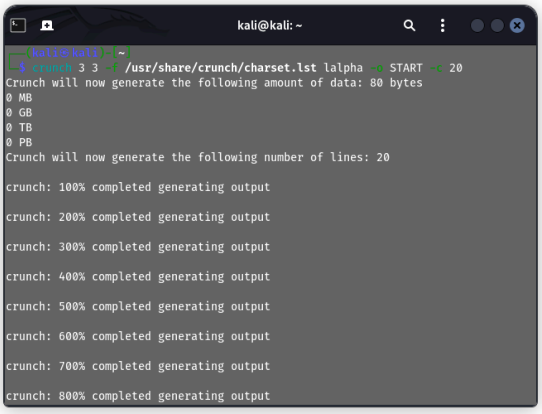
```
crunch 6 6 0123456789 -b 1mb -o START
```

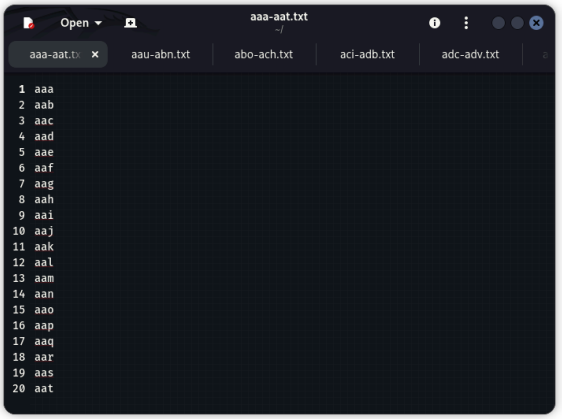




Specify the number of words

```
crunch 3 3 -f /usr/share/crunch/charset.lst lalpha -o START -c 20
```

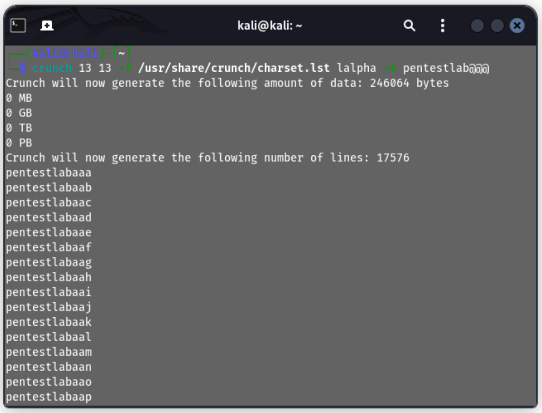


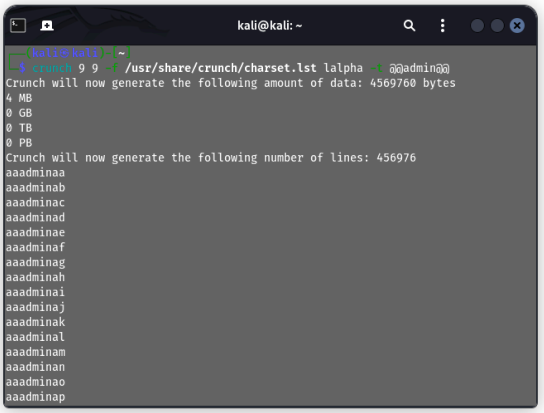


Prefix Wordlists

```
crunch 13 13 -f /usr/share/crunch/charset.lst lalpha -t pentestlab@@@
```

```
crunch 9 9 -f /usr/share/crunch/charset.lst lalpha -t @@admin@@
```



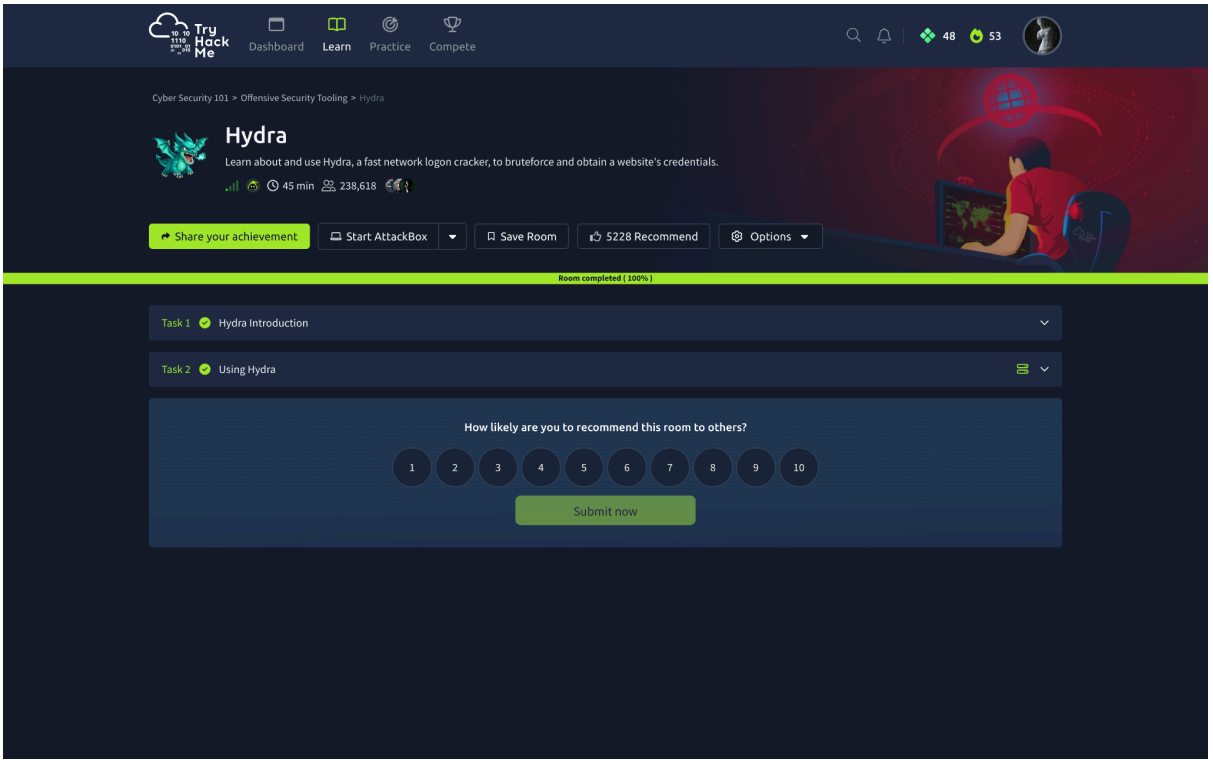


Learning to use Hydra

Hydra has the ability to bruteforce the following protocols:

```
Asterisk, AFP, Cisco AAA, Cisco auth, Cisco enable, CVS, Firebird, FTP,
HTTP-FORM-GET, HTTP-FORM-POST, HTTP-GET, HTTP-HEAD, HTTP-POST,
HTTP-PROXY, HTTPS-FORM-GET, HTTPS-FORM-POST, HTTPS-GET, HTTPS-HEAD,
HTTPS-POST, HTTP-Proxy, ICQ, IMAP, IRC, LDAP, MS-SQL, MYSQL, NCP, NNTP,
Oracle Listener, Oracle SID, Oracle, PC-Anywhere, PCNFS, POP3, POSTGRES,
RDP, Rexec, Rlogin, Rsh, RTSP, SAP/R3, SIP, SMB, SMTP, SMTP Enum, SNMP
v1+v2+v3, SOCKS5, SSH (v1 and v2), SSHKEY, Subversion, Teamspeak (TS2),
Telnet, VMware-Auth, VNC and XMPP.
```

Kali Hydra tool page: https://en.kali.tools/?p=220



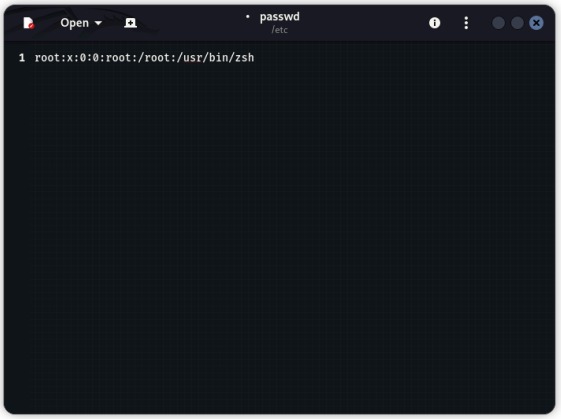
- ☑ Complete the 'Hydra' room on THM

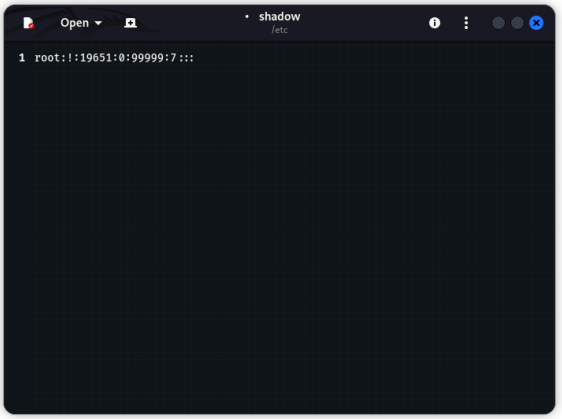
Cracking windows and linux password hashes

Cracking Linux Password Hashes using John the Ripper

- ☑ On your kali terminal change to the root directory and view all the files and folders.
- ☑ Switch to the /etc directory and look for the “passwd” and “shadow” files. The next step is to read the /etc/passwd file which contains all the accounts of the remote system.

/etc/passwd	/etc/shadow





In /etc/passwd the ':' (colon) is used separate fields.
The passwd file presents information about the users (in the order shown below).

- Username.
- Password. (using an x to indicate its been shadowed)
- User ID. (root will always be 0)
- Group ID.
- Comment (usually the users full name, or about their responsibilities).
- Users home directory.
- The users shell (Usually is set as a default of /bin/bash or /bin/sh)

In /etc/shadow the ':' (colon) is used separate fields.
The shadow file presents information about the users (in the order shown below).

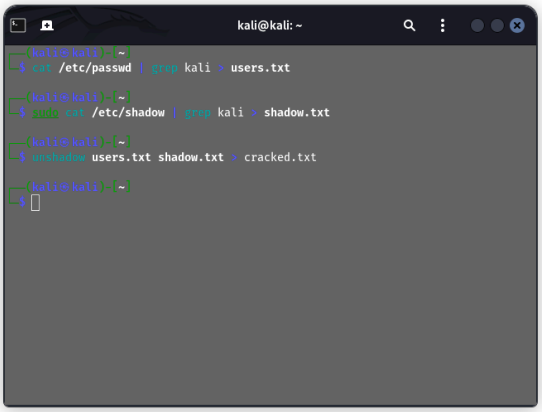
- Username.
- Encrypted password.
- Number of days since January 1st 1970, that the password was last changed.
- Number of days before the user is allowed to change their password.
- Number of days before the user must change their password.
- How many days in advance the user is warned of a password change.
- Number of days remaining for the user to change their password (else their account is disabled).
- A reserved field.

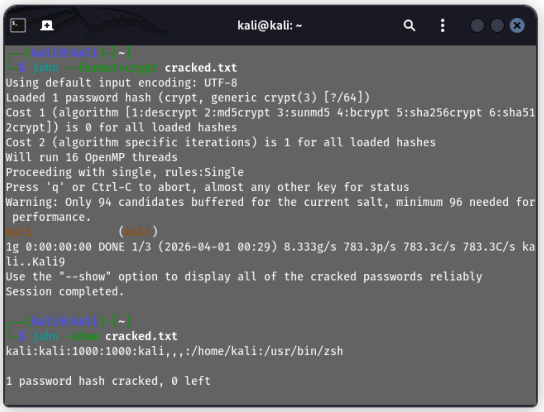
Digital forensics & Cyber Security

- [x] Copy passwd and shadow files to users.txt and shadow.txt

```
cat /etc/passwd | grep kali > users.txt
cat /etc/shadow | grep kali > shadow.txt
unshadow users.txt shadow.txt > cracked.txt

john --format=crypt cracked.txt
john -show cracked.txt
```

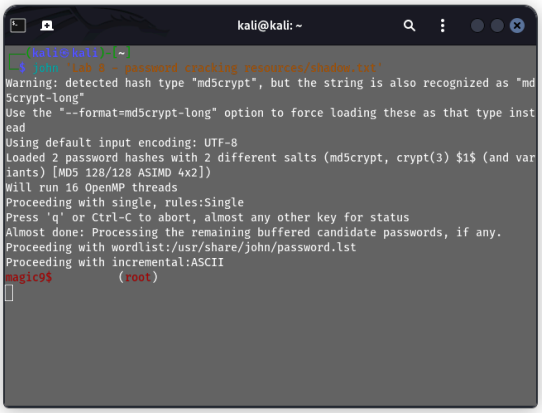


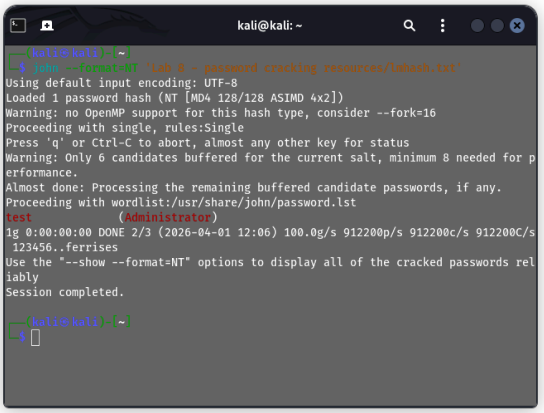


John generally has three modes - **single**, **wordlist** (with rules) & **incremental**,
and using the command *john cracked.txt* will use those modes in that order.

- [x] Create some test users on your own kali machine, give them all fairly easy short passwords, and then view the passwd and shadow files to see these users. You can use the commands *useradd user1* and *passwd user1* to create these users. To remove these users use *userdel –r user1*
- [x] Unshadow your own passwd and shadow files and try to crack the passwords with john.
- [x] Use John to try to crack the shadow file that is included on the resources.

```
shadow.txt                                  lmhash.txt
```





Cracking windows LM Hashes and SAM file

Understanding Windows Password Hashes

LM Hash (Legacy)

- **Old format** (pre-Windows Vista)
- **Weaknesses:** Case-insensitive, split into two 7-char halves, DES-based
- **Format:** `USERNAME:RID:LMHASH:NTHASH:::` (in SAM dump format)
- **How to crack:** LM hashes are now trivial — online rainbow tables work instantly

NTLM Hash (Modern)

- **Used:** Windows NT and later
- **Stronger:** Case-sensitive, MD4 algorithm, no length limitation
- **Format:** 32 hex characters
- **Example:** `aad3b435b51404eeaad3b435b51404ee` (empty hash)

SAM File

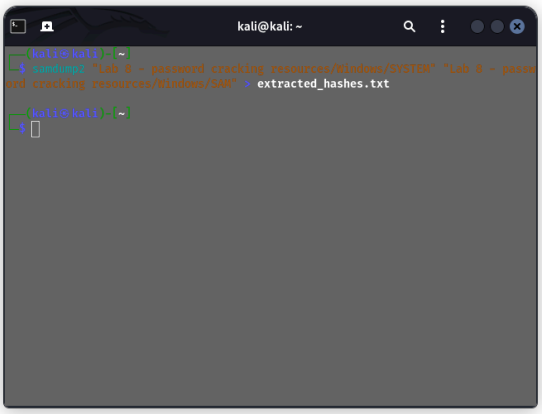
- **Location:** `C:\Windows\System32\config\SAM`
- **What it contains:** Local user accounts and password hashes (NTLM, LM)
- **Companion file:** SYSTEM registry hive (contains boot key for decryption)
- **Access:** Locked while Windows is running — extract from offline/backup

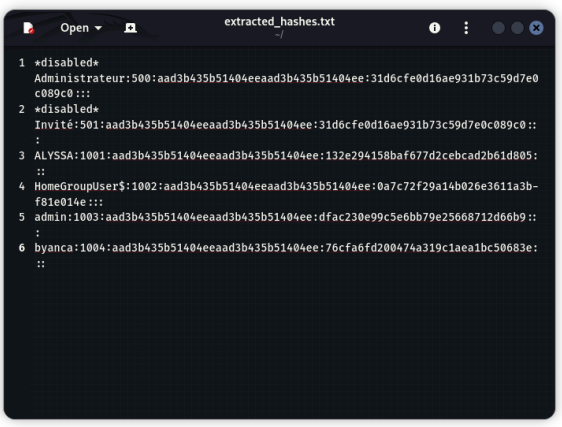
Extracting Hashes from SAM/SYSTEM Files

On Kali: Using `samdump2` or `pwdump`

Using samdump2

```
samdump2 SYSTEM SAM > extracted_hashes.txt
```



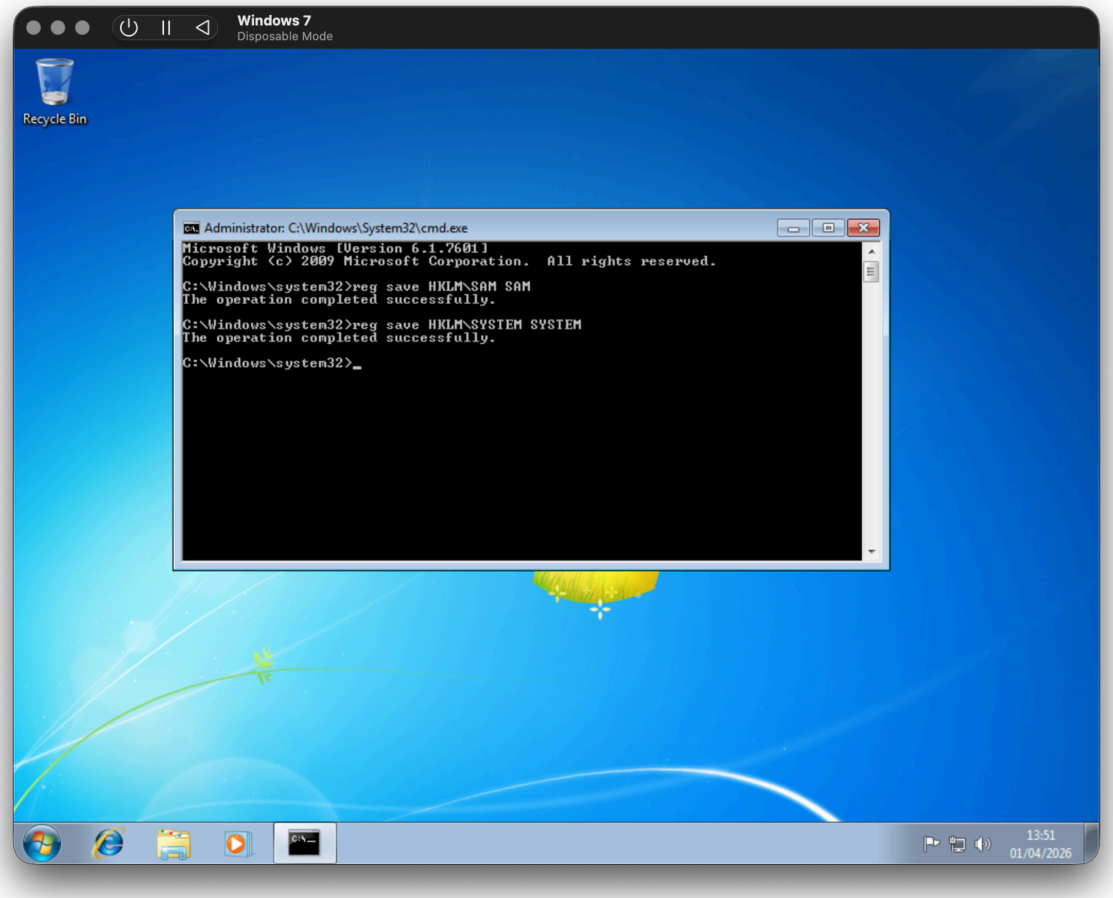


This decrypts the SAM file using the SYSTEM hive and outputs in format:

```
Username:RID:LMHash:NTHash:::
Administrator:500:aad3b435b51404eeaad3b435b51404ee:8846f7eaee8fb117ad06b
dd830b7586c:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:aad3b435b51404eeaad3b435b5140
4ee:::
User1:1000:e52cac67419a6a9a42f8b3674a46f028:5f4dcc3b5aa765d61d8327deb882
cf99:::
```

On Windows: Using `reg save` (before exfil)

```
reg save HKLM\SAM SAM
reg save HKLM\SYSTEM SYSTEM
```



Parsing the Hash Format

Once we have the dump, it'll looks like:

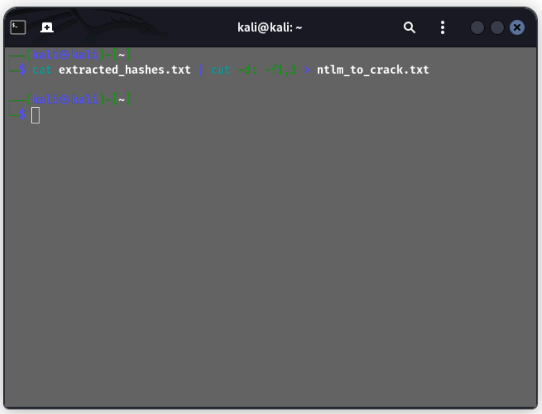
```
Username:RID:LMHash:NTHash:::
```

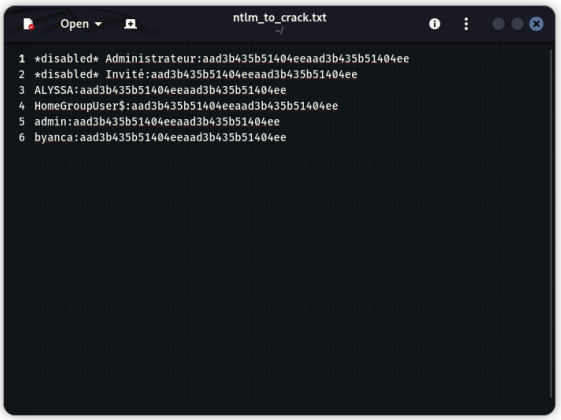
What each field means:

- `Username`: Account name
- `RID`: Relative ID (500=Admin, 501=Guest, 1000+=regular users)
- `LMHash`: Legacy LM hash (all zeros if disabled)
- `NTHash`: NTLM hash (the one to crack)
- `::::`: Legacy fields (usually empty)

Extract just the NTLM hash for cracking:

```
# Extract username:nthash pairs
cat extracted_hashes.txt | cut -d: -f1,3 > ntlm_to_crack.txt
```





Cracking LM Hashes (Quick Win)

Option 1: Online (instant)

- Visit: https://crackstation.net/
- Paste the LM hash
- Result appears in seconds

Hash	Type	Result
aad3b435b51404eeaad3b435b51404ee	LM	

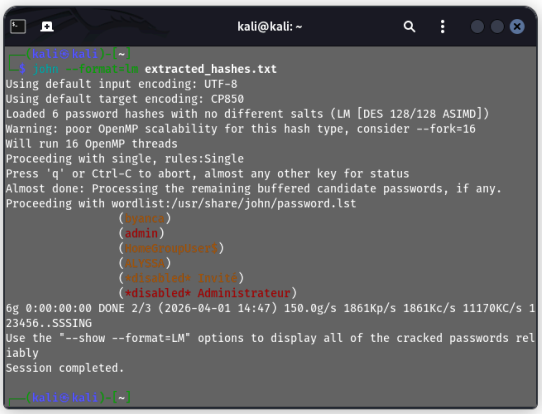
Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

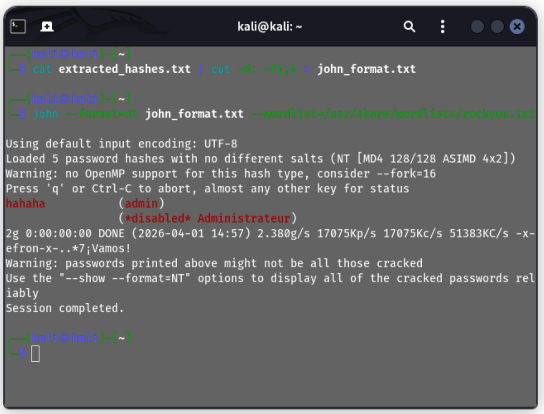
Hash	Type	Result
dfac230e99c5e6bb79e25668712d66b9	NTLM	hahaha

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Option 2: Local with John

```
john --format=lm extracted_hashes.txt
john --show extracted_hashes.txt
```





Why LM is weak:

- Only 56-bit key strength per 7-char block
- Uppercase converted to lowercase
- Pre-computed rainbow tables = instant crack

Cracking NTLM Hashes with John

Basic John approach:

```
# Format the hashes first
cat extracted_hashes.txt | cut -d: -f1,3 > john_format.txt
# john_format.txt should look like:
# Administrator:8846f7eaee8fb117ad06bdd830b7586c
# User1:5f4dcc3b5aa765d61d8327deb882cf99

# Crack with wordlist
john --format=nt john_format.txt
--wordlist=/usr/share/wordlists/rockyou.txt

# Or use incremental mode (slower but thorough)
john --format=nt john_format.txt --incremental

# View results
john --show john_format.txt
```

Common NTLM hashes you might encounter:

```
Empty password:  31d6cfe0d16ae931b73c59d7e0c089c0
"password":      5f4dcc3b5aa765d61d8327deb882cf99
"admin":         209c6174da490caeb422f3fa5a7ae634
```

Introduction to Hashcat

Hashcat Basics

Hashcat is much faster than John (especially with GPU) but has a steeper learning curve.

Key Advantages

- GPU-accelerated (100x+ faster than CPU for some hash types)
- Supports 300+ hash algorithms
- More efficient memory management
- Better for large-scale cracking

Hashcat Hash Types

Common hash type IDs (use with `-m`):

```
900  = MD4
1000 = NTLM (Windows hashes) ★
1100 = Domain Cached Credentials (DCC)
1400 = SHA256($pass.$salt)
3000 = LM hash
5500 = NetNTLMv2
```

Hashcat Syntax

Basic structure:

```
hashcat -m [hash_type] -a [attack_mode] [hashfile] [wordlist]
```

Attack Modes (`-a`)

- `0` = Straight (wordlist)
- `1` = Combination (word1 + word2)
- `3` = Brute force (mask)
- `6` = Hybrid wordlist + mask
- `7` = Hybrid mask + wordlist

Hashcat vs John Comparison

Feature	John	Hashcat
Speed	CPU-based, moderate	GPU-based, very fast
Ease of use	Simpler syntax	More complex syntax
Hash support	~20+ types	300+ types
GPU support	Limited	Excellent
Best for	Quick cracks, small hashes	Large-scale, GPU-enabled
Wordlist modes	Single, incremental, rules	Single, combo, rules, brute

Hashcat Tips & Tricks

Monitor progress:

```
hashcat -m 1000 -a 0 hashes.txt rockyou.txt --status
# Shows speed (H/s), progress, ETA

# Pause: press 's' during execution
# Resume: hashcat --restore
```

Resume cracked passwords:

```
# Hashcat saves state automatically
# Resume interrupted session
hashcat --restore

# Show cracked passwords
hashcat -m 1000 hashes.txt --show
```

Benchmark your system:

```
hashcat -m 1000 --benchmark
# Tells you H/s (hashes/second) your system can achieve
```

Custom mask patterns:

```
?a = [0-9a-zA-Z!@#$%...]  (all printable)
?u = [A-Z]                (uppercase)
?l = [a-z]                (lowercase)
?d = [0-9]                (digits)
?s = [!@#$%^&*()...]      (special chars)
?h = [0-9a-f]             (hex)
?x = [0-9a-fA-F]          (hex uppercase)

# Examples:
?u?l?l?l?l?d?d    = Passw0rd (variation)
?d?d-?l?l-?l?l    = 12-ab-cd (pattern)
```

Cracking Linux Shadow File (MD5-crypt) Hashcat (shadow.txt)

Cracking the same files that we used with John, to compare john and hashcat. For Linux hashes, John the Ripper is often faster than Hashcat

```
root:$1$CQoPk7Zh$370xDLmeGD9m4aF/ciIlC.:14425:0:99999:7:::
```

Hash format breakdown:

```
$1$ = MD5-crypt (Unix/Linux)
CQoPk7Zh = Salt (8 characters)
370xDLmeGD9m4aF/ciIlC. = Hash (22 characters)
```

Extract Only Crackable Hashes

```
# Extract crackable accounts (those with $1$ or $2$ or $5$ or $6$)
grep -E '\$[1256]\$' shadow.txt > shadow_crackable.txt

cat shadow_crackable.txt
# Output:
# root:$1$CQoPk7Zh$370xDLmeGD9m4aF/ciIlC.:14425:0:99999:7:::
# tintin:$1$wKAP1RyH$JeCAcEGhSGVlD0J7.AMg.0:14396:2:5:7:30::
```

Extract Hash-Only Format for Hashcat

Hashcat wants format: *username:hash* OR just *hash*

Option 1: Extract with username (better for tracking)

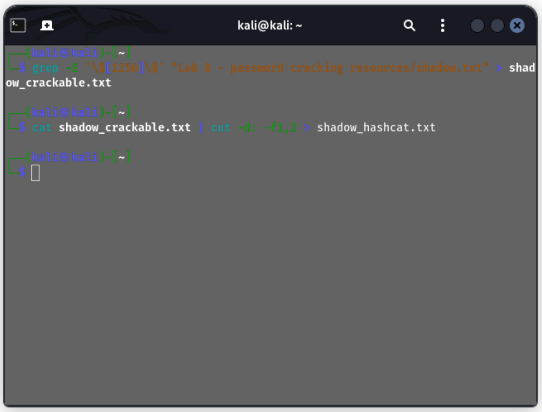
```
# Extract username:hash pairs
cat shadow_crackable.txt | cut -d: -f1,2 > shadow_hashcat.txt

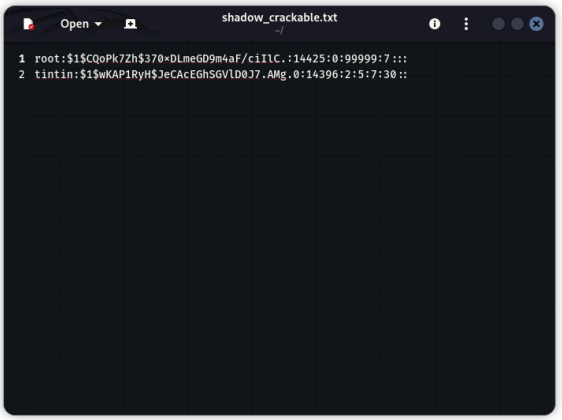
cat shadow_hashcat.txt
# Output:
# root:$1$CQoPk7Zh$370xDLmeGD9m4aF/ciIlC.
# tintin:$1$wKAP1RyH$JeCAcEGhSGVlD0J7.AMg.0
```

Option 2: Just hashes (if you don't need usernames)

```
cat shadow_crackable.txt | cut -d: -f2 > shadow_hashes_only.txt

cat shadow_hashes_only.txt
# Output:
# $1$CQoPk7Zh$370xDLmeGD9m4aF/ciIlC.
# $1$wKAP1RyH$JeCAcEGhSGVlD0J7.AMg.0
```





Crack with Hashcat

```
# Basic wordlist attack (hash type 500 = MD5-crypt)
hashcat -m 500 -a 0 shadow_hashcat.txt /usr/share/wordlists/rockyou.txt
--username

# With progress display
hashcat -m 500 -a 0 shadow_hashcat.txt /usr/share/wordlists/rockyou.txt
--username --status

# Brute force 6-char lowercase (WARNING: slow for crypt hashes)
# This will take a VERY long time
hashcat -m 500 -a 3 shadow_hashcat.txt ?l?l?l?l?l?l

# View results
hashcat -m 500 shadow_hashcat.txt --show --username
```

Windows LM/NTLM Hash (lmhash.txt)

```
Administrator:500:01FC5A6BE7BC6929AAD3B435B51404EE:0CB6948805F797BF2A828
07973B89537:::
```

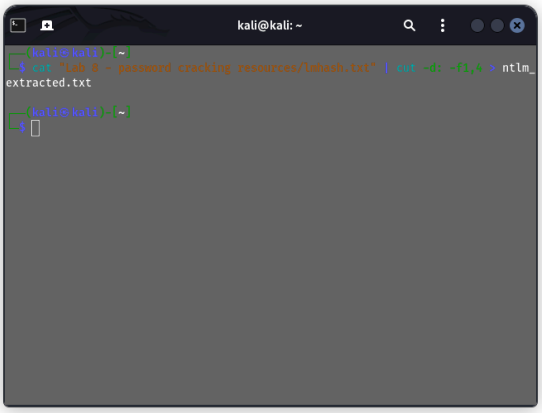
Break it down:

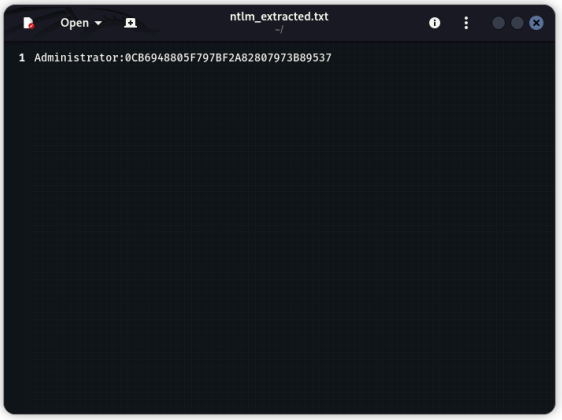
```
Administrator = Username
500 = RID (500 = Administrator account)
01FC5A6BE7BC6929AAD3B435B51404EE = LM Hash (OLD, breakable)
0CB6948805F797BF2A82807973B89537 = NTLM Hash (Modern, crackable)
```

Extract NTLM Hash

```
# Extract just username:nthash
cat lmhash.txt | cut -d: -f1,4 > ntlm_extracted.txt

cat ntlm_extracted.txt
# Output:
# Administrator:0CB6948805F797BF2A82807973B89537
```



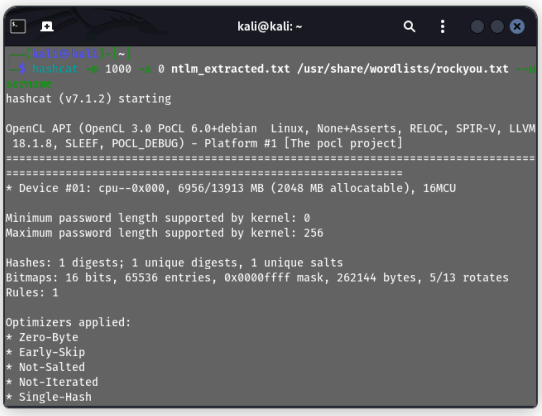


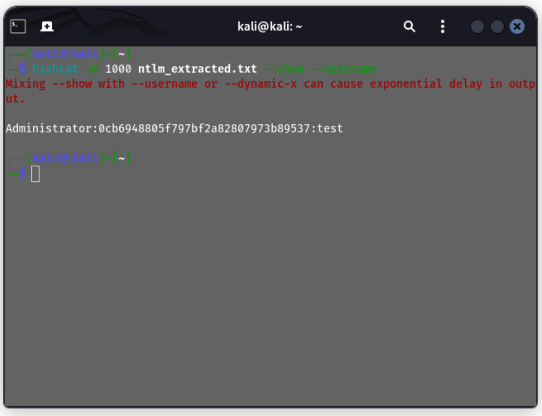
Crack NTLM with Hashcat

```
# Wordlist attack (FAST - should crack in seconds/minutes)
hashcat -m 1000 -a 0 ntlm_extracted.txt /usr/share/wordlists/rockyou.txt
--username

# Brute force 8-char mixed case + numbers (faster than Linux hashes)
hashcat -m 1000 -a 3 ntlm_extracted.txt ?l?l?l?l?l?l?l?l

# View results
hashcat -m 1000 ntlm_extracted.txt --show --username
```





Cracking NTLM (SAM, SYSTEM) with Hashcat

Prepare hashes (format: one hash per line, or `username:hash`):

```
# Create hashes.txt
echo "8846f7eaee8fb117ad06bdd830b7586c" > hashes.txt
echo "5f4dcc3b5aa765d61d8327deb882cf99" >> hashes.txt

# Or with usernames (hashcat ignores username part)
echo "Administrator:8846f7eaee8fb117ad06bdd830b7586c" > hashes.txt
```

Attack with wordlist:

```
hashcat -m 1000 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

Brute force (mask attack):

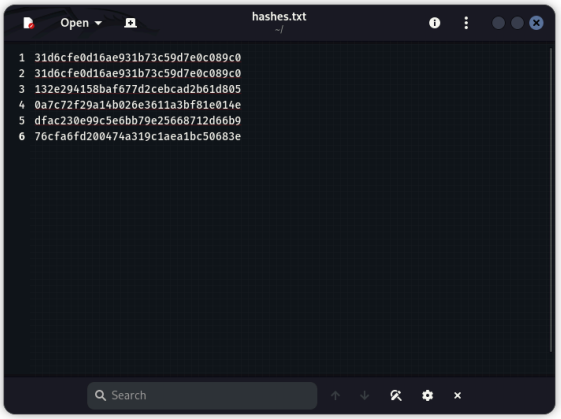
```
# Crack 6-char passwords with lowercase letters + numbers
# ?a = all printable chars, ?l = lowercase, ?d = digit, ?u = uppercase
hashcat -m 1000 -a 3 hashes.txt ?l?l?l?l?d?d
# This tries: aaaaa00, aaaaaa01, ... zzzzz99

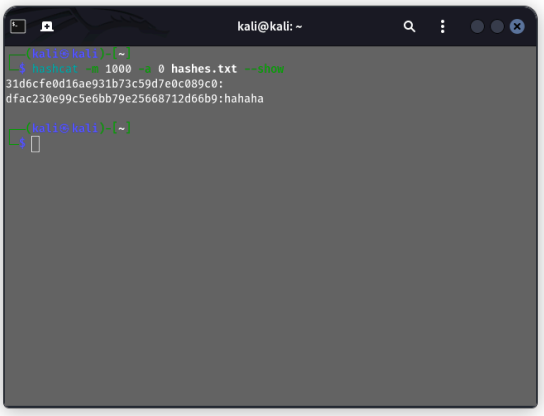
# 8-char passwords (lowercase only)
hashcat -m 1000 -a 3 hashes.txt ?l?l?l?l?l?l?l?l
```

With rules (modify wordlist):

```
hashcat -m 1000 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt \
  -r /usr/share/hashcat/rules/best64.rule

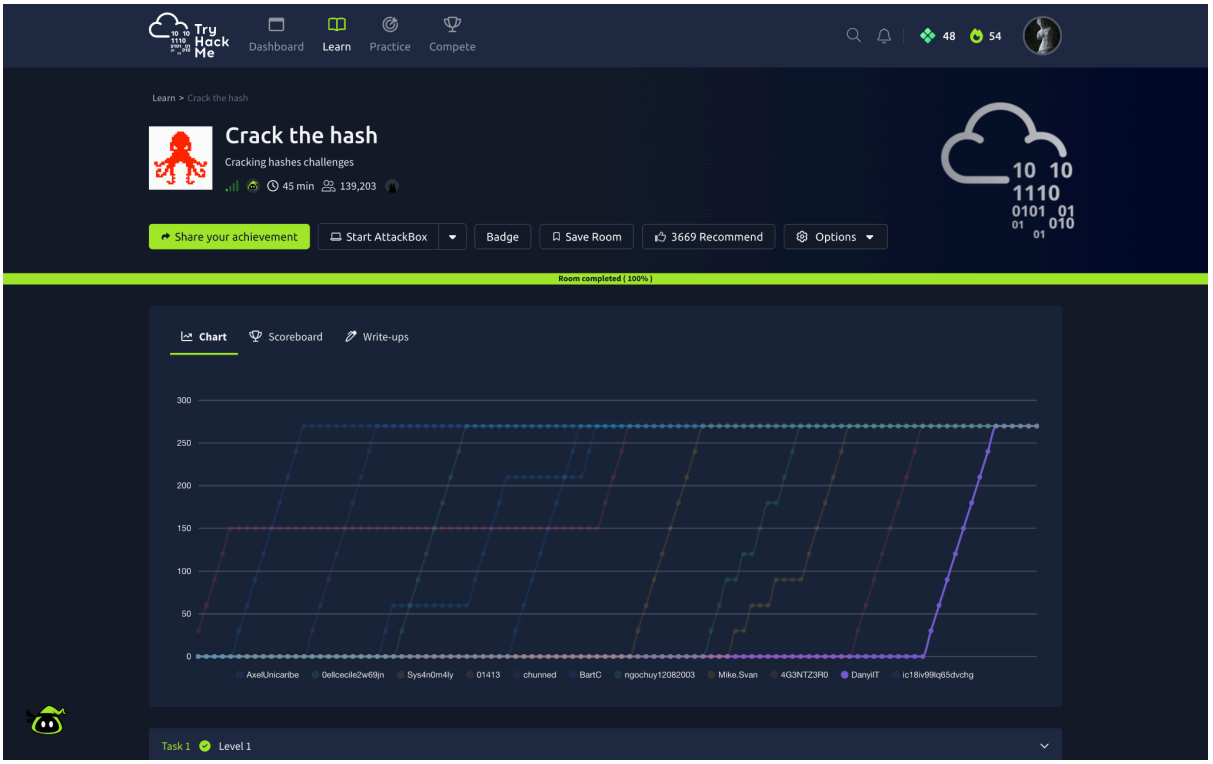
# This applies transformations like:
# password -> Password
# password -> password!
# password -> p@ssw0rd
```





THM hash cracking rooms

THM rooms to complete:



☑ Crack the Hash



☑ Crack the Hash Level 2